

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores



Fundamentos de Arquitectura de Computadores

Proyecto grupal 1

Bitácora

Máquinas de estados y protocolos de comunicación SPI

Estudiante	Carné
Bolaños Barboza Gabriel	2022327511
Rodríguez Rojas Andrés	2019279722
Somarribas Montero Isaac	2020125516
Soto Varela Óscar	2020092336

Profesor:

Luis Alberto Chavarría Zamora

27 de mayo de 2025

Proyecto Grupal 1

Máquinas de estados y protocolo de comunicación SPI

Fecha de asignación: 22 abril 2025
Grupos: 3-4 personas

Fecha de entrega: 20 mayo 2025
Profesores: Luis Chavarría Zamora

Mediante el desarrollo de este proyecto, los estudiantes aplicarán los conceptos de arquitectura de computadores en el diseño e implementación en hardware.
Atributo relacionados al presente trabajo: Conocimiento de Ingeniería (CI), el cual se encuentra en Avanzado (M).

1. Descripción General: SPI

El SPI (Serial Peripheral Interface) es un protocolo de comunicación síncrona utilizado para la transferencia de datos entre dispositivos electrónicos, como microcontroladores, sensores, y periféricos. El SPI se basa en una arquitectura maestro-esclavo, donde un dispositivo maestro controla la comunicación con uno o varios dispositivos esclavos. Aquí hay una breve explicación de sus principales líneas de entrada/salida (I/O):

1. MISO (Master In Slave Out): Esta línea es utilizada por el dispositivo esclavo para enviar datos al dispositivo maestro. Contiene la información que el dispositivo maestro necesita recibir del dispositivo esclavo.
2. MOSI (Master Out Slave In): La línea MOSI se utiliza para transmitir datos desde el dispositivo maestro hacia el dispositivo esclavo. Los datos generados por el dispositivo maestro se envían a través de esta línea al dispositivo esclavo.
3. SCLK (Serial Clock): La línea de reloj serial (SCLK) es controlada por el dispositivo maestro y se utiliza para sincronizar la comunicación entre el maestro y el esclavo. Los datos se transmiten y se reciben sincronizados con el pulso de reloj en esta línea.
4. SS/CS (Slave Select/Chip Select): Esta línea se utiliza para habilitar o seleccionar un dispositivo esclavo específico con el que se desea comunicar el dispositivo maestro. Puede haber múltiples líneas SS/CS si se comunican con varios dispositivos esclavos.

1.1. PWM

El PWM, o Modulación por Ancho de Pulso (por sus siglas en inglés, Pulse Width Modulation), es una técnica utilizada en la electrónica y la ingeniería digital para controlar la potencia entregada a dispositivos como motores, luces LED y otros componentes eléctricos. Funciona generando una señal de salida digital que varía en su ancho de pulso (duración) a lo largo del tiempo, mientras que la frecuencia de la señal se mantiene constante.

En PWM, el ciclo de trabajo, también conocido como duty cycle, representa la proporción de tiempo durante el cual la señal está en estado alto (encendido) en relación con el período total de la señal. Por ejemplo, si el ciclo de trabajo es del 50 %, significa que la señal está en estado alto durante la mitad del período y en estado bajo durante la otra mitad. Esto permite controlar la potencia suministrada al dispositivo conectado, ya que un ciclo de trabajo más alto significa más potencia y viceversa.

El PWM se utiliza comúnmente para controlar la velocidad de motores, la intensidad de luces LED o la posición de servomotores. Al ajustar el ciclo de trabajo de la señal PWM, se puede lograr un control preciso sobre el dispositivo conectado, permitiendo variar su rendimiento de manera efectiva y eficiente.

Un ejemplo de esto se observa en la Figura 1.

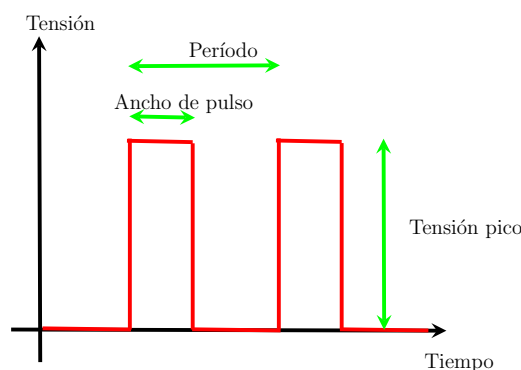


Figura 1: Diagrama de tiempo del PWM.

1.2. Handshake

Un "*handshake*" (apretón de manos, en inglés) es un término utilizado en tecnología para describir el proceso de inicio de una comunicación entre dos dispositivos o sistemas, como en una conexión de red o SPI. A continuación se muestra una breve explicación de cómo funciona:

- **Inicio:** Cuando dos dispositivos desean comunicarse, el proceso de handshake comienza con un dispositivo (llamado el iniciador) enviando una solicitud de conexión al otro dispositivo (llamado el receptor). Esta solicitud generalmente contiene información sobre los parámetros de la conexión que se va a establecer.
- **Respuesta:** El dispositivo receptor recibe la solicitud y responde con una confirmación. Esta respuesta indica que está listo para establecer la conexión y también puede incluir información adicional sobre los parámetros de la comunicación.

- Acuerdo: Ambos dispositivos, el iniciador y el receptor, deben acordar los detalles de la conexión, como el tipo de protocolo a utilizar, la velocidad de transmisión de datos y otros parámetros importantes. Este acuerdo asegura que ambos dispositivos estén en la misma página antes de comenzar la comunicación.
- Comunicación: Después de que se ha establecido el acuerdo, los dispositivos pueden comenzar a intercambiar datos. Ahora están sincronizados y listos para transmitir información de manera eficiente.

El proceso de "handshake" garantiza que la comunicación entre los dispositivos sea confiable y que ambos extremos estén preparados para la transferencia de datos. Es fundamental en diversas tecnologías, como en las conexiones de red, las comunicaciones Bluetooth y muchas otras aplicaciones donde la confiabilidad y la sincronización son importantes.

2. Especificación

Se desarrollará un sistema de control de velocidad con ALU (16 velocidades diferentes) (incrementaría la cantidad de bits con respecto al proyecto individual a 4 bits) mediante PWM para un motor de DC (debe existir desacople sino se quemaría la FPGA). Esto se observa en la Figura 2.

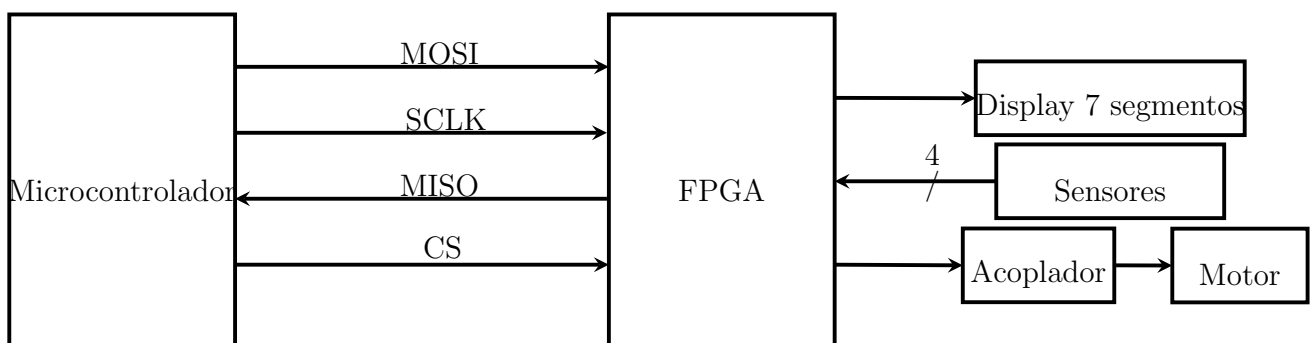


Figura 2: Propuesta de diseño para el sistema.

El sistema estará conformado por un Arduino, una [FPGA Terasic DE10-Nano](#) o bien la misma de Taller de Diseño Digital, un motor de DC (debe existir desacople sino se quemaría la FPGA), un display de siete segmentos y un tablero de sensores como el proyecto individual (solo ocupan detectar los sensores). Para cada una de las etapas se tiene lo siguiente:

2.1. Microcontrolador

En el microcontrolador los estudiantes definirán protocolo basado en SPI para transmitir a la FPGA. El microcontrolador será el master de esta operación (**reemplazará el acumulado**). Debe mostrar en consola (computadora personal) el mensaje enviado a la FPGA y el mensaje de *Acknowledgement* recibido desde la FPGA. Deben establecer de que forma se generarán los patrones para hacer variaciones de la velocidad. Se tendrán 16 velocidades, desde 0 (motor detenido) hasta su máxima velocidad. **El microcontrolador debe iniciar la comunicación y después pasar un número de 0 a F .**

2.2. FPGA

El circuito estructural (**deben usar tablas de transición y salida como las vistas en clase, si lo realizan de forma comportamental tendrán cero**) armado en la FPGA servirá como Slave del Arduino. Dentro de la FPGA se realizará el sistema estructuralmente, es decir, solo mediante ecuaciones booleanas y variables de registro, no se pueden usar estructuras de alto nivel como *cases*, *if*, tampoco se puede usar el operador ternario (?). **Si no se puede implementar solo con compuertas, no se puede usar.** La FPGA se interfazará con un desacople para controlar la velocidad del motor mediante un PWM, al mismo tiempo mostrará en un *display* de siete segmentos un valor normalizado de velocidad (desde 0 hasta F circular), deben usar una representación en hexadecimal (incrementaría la cantidad de bits con respecto al proyecto individual a 4 bits).

Dentro de la FPGA se implementará una *ALU* que recibe la siguiente entrada:

- Cuatro sensores, que luego representan un número de dos bits (como en el proyecto individual).
- Un acumulado seteado por el microcontrolador con cuatro bits.
- Resultado de la ALU de cuatro bits.
- Switches de la FPGA para escoger la operación.

Deben ejecutar las medidas mecánicas necesarias para asegurar el orden y estabilidad de los elementos durante la presentación.

Se debe establecer un handshake inicial con el Arduino mediante SPI. Cuando recibe el mensaje del Arduino, deben generar un acknowledgement para que el Arduino sepa que ya lo ejecutó y sino para volverlo a enviar (**si no se logra esto, no realiza nada el sistema**).

Este módulo se debe interfazar además con lo siguiente:

- Motor de DC: Por razones eléctricas se necesita un acople para conectarlo.
- Un display de siete segmentos (puede ser de la FPGA).

Dentro de la FPGA se deberá construir un PWM para controlar la velocidad del motor de DC en 16 velocidades y también una ALU con las siguientes operaciones:

1. Multiplicación.
2. Resta.
3. AND.
4. XOR.

Tome en cuenta que además debe mostrar las banderas mediante LEDs. **El microcontrolador no envía la operación resuelta, envía solo el valor en el mensaje.**

3. Metodología de trabajo

El proyecto debe seguir los siguientes aspectos de desarrollo colaborativo en git, sino, la parte funcional no será calificada y obtendrá nota de cero:

1. Utilice una cuenta de repositorio gratuita.
2. Cree un repositorio con el siguiente nombre: `<user_id>_compu_archi_found_2G1_2024`.
3. El `user_id` estará compuesto por la primera letra del nombre y el apellido de quien haga el repositorio.
Se usará un repositorio por grupo, el generador le realiza la invitación a las demás personas.
4. El repositorio debe ser privado. Proporcione acceso a `compu-archi-found-2S2024` (GitHub).
5. El repositorio de Git contendrá dos ramas principales: `master` y `development`.
6. Inicialmente, la rama de `development` se crea a partir del `master`.
7. Al trabajar en un proyecto, los estudiantes deben crear una nueva rama de trabajo desde `develop` y cuando la función esté lista, la rama debe fusionarse para `develop`. Cualquier corrección o modificación adicional después de `merge` debería requerir que se repita el proceso (es decir, crear la rama desde `develop` y fusionar los cambios más tarde). Una vez

que el código de desarrollo esté listo, se fusionará con **master** y se debe crear una **tag**. El proceso se describe en la siguiente Figura 3.

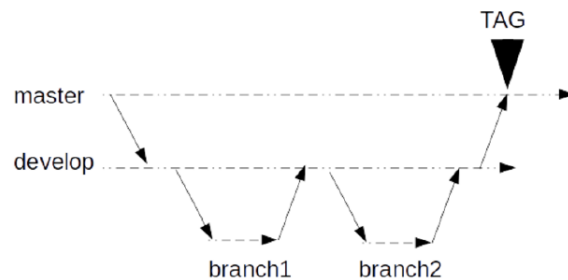


Figura 3: *Git workflow*

Adicionalmente se coloca este [enlace recomendado](#).

8. Después de haber realizado algunos proyectos la rama **master** debe verse así:

- master/
 - proyecto_1
 - proyecto_2
 - ...
- ...

Donde cada directorio de **proyecto_x** contiene todos los entregables para cada proyecto.

No es permitido realizar todo el trabajo en un solo commit, es decir, que realice el trabajo de forma local y solo suba el último entregable en el repositorio. Si no, obtendrá nota de cero. Debe mostrar avance incremental durante todas las semanas (se revisarán estadísticas). Se revisará el trabajo en equipo.

4. Evaluación y entregables

Se programarán horas de defensa para el mismo día de la entrega de documentación. Todos los archivos (incluyendo código fuente) serán entregados a las 11:59 pm ese mismo día (**realícenlo progresivamente y no lo dejen para el final**). Si entregan después de esa hora, se rebajará un punto por minuto. **Si algo no queda claro o no sabe, no lo asuma, pregúntele al profesor.** Como recomendación se presenta el cronograma de trabajo de la Tabla 1.

La evaluación del proyecto se da bajos los siguientes rubros contra rúbrica correspondiente:

Tabla 1: Cronograma sugerido para el proyecto

Semana	Actividades sugeridas
1	Estudio del problema y conceptos como SPI, PWM, FPGA y handshake. Consignación en bitácora.
2	Propuesta de solución general modularizada y diseños iniciales. Consignación en bitácora.
3	Simulación y pruebas iniciales en la PFPGA de los módulos. Consignación en bitácora.
4	Integración de las soluciones en la FPGA y el Arduino. Consignación en bitácora.
5	Validación de las soluciones. Consignación en bitácora.

- Presentación proyecto 100 % funcional (70 %): La defensa se realizará de **forma presencial** en un horario a definir. La defensa se debe realizar en un espacio de **media hora** el diseño sintetizable y ejecutado en la tarjeta **FPGA Terasic DE10-Nano**. Todos los integrantes deben intervenir en la defensa. Adicionalmente se deben mostrar simulaciones y el diseño que está siendo ejecutado. **No traer sintetizado el diseño en la FPGA.**

Los entregables adicionales que se revisarán **durante** la defensa son los siguientes:

1. Diagrama general de todo el sistema, desde el usuario hasta los módulos.
 2. Diagrama general de la FPGA, debe ser diseñado por los estudiantes, no será válido presentar el que genera el software de FPGA.
- Bitácora (Atributo evaluado: Conocimiento de Ingeniería) (15 %): Es un documento donde los estudiantes indica detalladamente los procedimientos que implementaron en el diseño, tablas, ecuaciones booleanas, herramientas de automatización utilizadas y otras relacionadas. Debe existir una división clara de días en que se realizó. Los estudiantes deben desarrollarlo en **Overleaf**¹ y compartir con solo lectura al profesor del curso. El documento debe crecer en contenido conforme avanza el tiempo, no anotar las actividades al final, y no anotarlo a más tardar de dos días de haber realizado la actividad.
 - Propuesta de investigación (Atributo evaluado: Investigación) (15 %): Este documento contará con las siguientes secciones donde se explique el proyecto en su totalidad:
 1. Identificación del problema complejo de ingeniería a investigar.
 2. Diseño de una propuesta de investigación.
 3. Ejecución de la metodología del plan de investigación para la obtención de datos relevantes.

¹Se solicita Overleaf pues cuenta con historial sencillo de seguir y es colaborativo.

4. Análisis los datos obtenidos en el desarrollo de la investigación.
5. Elaboración de conclusiones a partir de la síntesis y análisis de los resultados de la investigación.

Se seguirán los siguientes lineamientos:

1. Los documentos serán sometidos a control de plagios para eliminar cualquier intento de plagio con trabajos de semestres anteriores, actual o copias textuales, tendrán nota de cero los datos detectados. Se prohíbe el uso de referencias hacia sitios no confiables.
2. No coloque código fuente en los documentos, quita espacio y aporta poco. Mejor explique el código, páselo a pseudocódigo o use un diagrama.

I. BITÁCORA

Fecha	Miembro(s)	Actividad Realizada	Duración
16/05/2025	Todos	Reunión grupal vía Discord para distribuir tareas	1 hora
17/05/2025	Óscar	Desarrollo Comunicación UART sin utilizar compuertas	15 horas
19/05/2025	Andrés Rodríguez	Inicio del modelo del BCD estructural con compuertas.	2 horas
19/05/2025	Isaac Somarribas	Investigación del funcionamiento del PWM	1 día
20/05/2025	Andrés Rodríguez	Adaptación del módulo de la comunicación UART para su funcionamiento con compuertas.	4 horas
20/05/2025	Óscar Soto	Inicio de la ALU	3 horas
21/05/2025	Gabriel Bolaños	Desarrollo y pruebas del módulo PWM con lógica estructural	4 días
21/05/2025	Isaac Somarribas	Desarrollo y pruebas del módulo PWM con lógica estructural	4 días
21/05/2025	Andrés Rodríguez	Cálculo y estructuración de las máquinas de estados finitos necesarias para el funcionamiento de la comunicación UART.	3 horas
21/05/2025	Andrés Rodríguez	Adaptación del módulo de la comunicación UART para su funcionamiento con compuertas.	6 horas
21/05/2025	Óscar Soto	Adaptación del módulo de la comunicación UART para su funcionamiento con compuertas.	6 horas
24/05/2025	Andrés Rodríguez	Finalización del modelo del BCD estructural con compuertas.	2 horas
26/05/2025	Andrés Rodríguez	Finalización de la ALU	5 horas
26/05/2025	Gabriel Bolaños	Finalización de la ALU	5 horas
26/05/2025	Andrés Rodríguez	Integración de los módulos de comunicación, ALU y BCD para un funcionamiento completo.	4 horas
26/05/2025	Gabriel Bolaños	Integración de los módulos de comunicación, ALU y BCD para un funcionamiento completo.	4 horas

II. DISEÑO E IMPLEMENTACIÓN

II-A. Comunicación UART - Estructuración de máquinas de estados finitos

Para el desarrollo del taller se implementó el diseño de tres máquinas de estados finitos (MEF), a continuación se presentan evidencias del proceso de diseño de las mismas.

Inicialmente se estableció una codificación para los estados de la máquina de estados finitos (IDLE, START_BIT, DATA_BITS, STOP_BITS, CLEANUP), como se observa en la figura 1.

	S ₂	S ₁	S ₀
IDLE	0	0	0
START_BIT	0	0	1
DATA_BITS	0	1	0
STOP_BITS	0	1	1
CLEANUP	1	0	0

Figura 1. Codificación: UART_RX.

De este modo se establece el diagrama de estados, junto con las condiciones que se requieren para el paso de un estado a otro, como se demuestra en la figura 2.

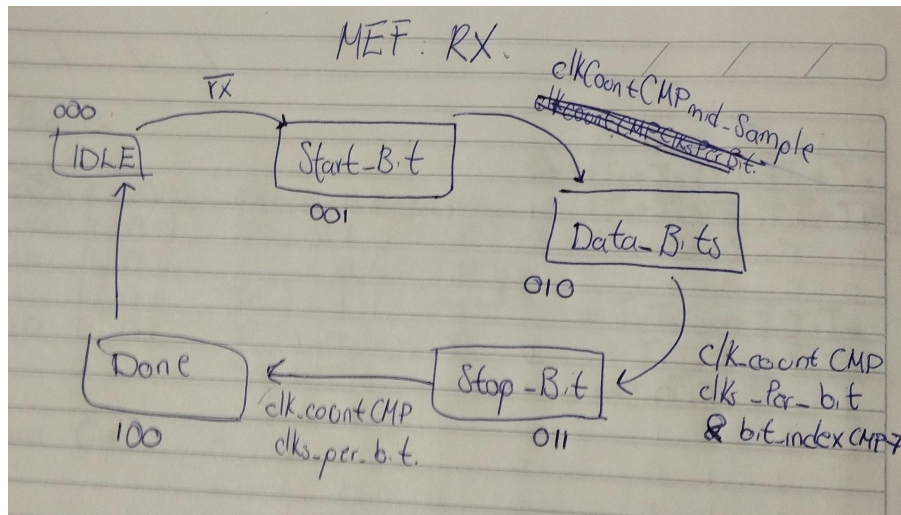


Figura 2. Diagrama de estados: UART_RX.

Finalmente, se detalla la tabla de estados y entradas (3) para la implementación de las salidas propias del siguiente estado ($state_next$), y haciendo uso del método de suma de productos, se establecen cada una de sus componentes ($state_next[1]$, $state_next[0]$).

state[2]	state[1]	state[0]	Inputs				next_state[2]	next_state[1]	next_state[0]
			rx_sync	clk_countCMPmid_sample	clk_countCMPclks_per_bit	bit_indexCMP7			
0	0	0	0	X	X	X	0	0	1
0	0	0	1	X	X	X	0	0	0
0	0	1	X	1	X	X	0	1	0
0	0	1	X	0	X	X	0	0	1
0	1	0	X	X	1	1	0	1	1
0	1	0	X	X	1	0	0	1	0
0	1	0	X	X	0	X	0	1	0
0	1	1	X	X	1	X	1	0	0
0	1	1	X	X	0	X	0	1	1
1	0	0	X	X	X	X	0	0	0

$$state_next[2] = \sim state[2] \& state[1] \& state[0] \& clk_countCMPclks_per_bit$$

$$state_next[1] = (\sim state[2] \& \sim state[1] \& state[0] \& clk_countCMPmid_sample) \vee (\sim state[2] \& state[1] \& \sim state[0] \& clk_countCMPclks_per_bit \& bit_indexCMP7) \vee (\sim state[2] \& state[1] \& \sim state[0] \& clk_countCMPclks_per_bit \& \sim bit_indexCMP7) \vee (\sim state[2] \& state[1] \& \sim state[0] \& \sim clk_countCMPclks_per_bit) \vee (\sim state[2] \& state[1] \& state[0] \& \sim clk_countCMPclks_per_bit);$$

$$state_next[0] = (\sim state[2] \& \sim state[1] \& \sim state[0] \& \sim rx_sync) \vee (\sim state[2] \& \sim state[1] \& state[0] \& \sim clk_countCMPmid_sample) \vee (\sim state[2] \& state[1] \& \sim state[0] \& clk_countCMPclks_per_bit \& bit_indexCMP7) \vee (\sim state[2] \& state[1] \& state[0] \& \sim clk_countCMPclks_per_bit)$$

Figura 3. Tabla de estados y entradas: UART_RX.

Utilizando un método similar al implementado para la máquina de estados finitos UART_RX, se establecen la correspondiente codificación, diagrama de estados y tabla de estados y entadas de las figuras 4 y 5 respectivamente.

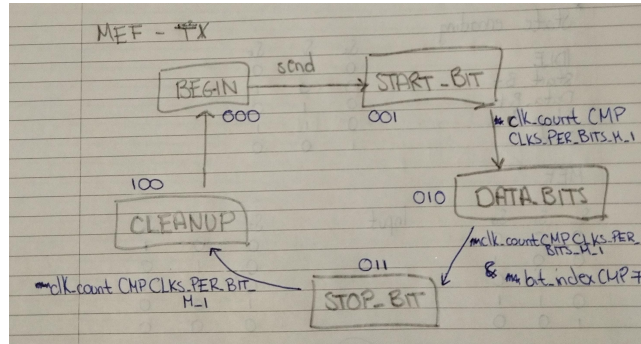


Figura 4. Diagrama de estados: UART_TX.

state[2]	state[1]	state[0]	Inputs			next_state[2]	next_state[1]	next_state[0]
			send	countCMPCLKS.PER.BIT	bit_indexCMP7			
0	0	0	1	X	X	0	0	1
0	0	0	0	X	X	0	0	0
0	0	1	X	0	X	0	1	0
0	0	1	X	1	X	0	0	1
0	1	0	X	0	0	0	1	1
0	1	0	X	1	X	0	1	0
0	1	0	X	X	1	0	1	0
0	1	1	X	0	X	1	0	0
0	1	1	X	1	X	0	1	1
1	0	0	X	X	X	0	0	0

$state_next[2] = (\sim state[2] \& state[1] \& state[0] \& \sim clk_countCMPCLKS_PER_BIT_M_1)$
 $state_next[1] = (\sim state[2] \& \sim state[1] \& state[0] \& \sim clk_countCMPCLKS_PER_BIT_M_1) \vee (\sim state[2] \& state[1] \& \sim state[0] \& \sim clk_countCMPCLKS_PER_BIT_M_1 \& \sim bit_indexCMP7) \vee (\sim state[2] \& state[1] \& state[0] \& clk_countCMPCLKS_PER_BIT_M_1) \vee (\sim state[2] \& state[1] \& \sim state[0] \& bit_indexCMP7) \vee (\sim state[2] \& state[1] \& state[0] \& clk_countCMPCLKS_PER_BIT_M_1)$
 $state[0] = (\sim state[2] \& \sim state[1] \& \sim state[0] \& send) \vee (\sim state[2] \& \sim state[1] \& state[0] \& clk_countCMPCLKS_PER_BIT_M_1) \vee (\sim state[2] \& state[1] \& \sim state[0] \& \sim clk_countCMPCLKS_PER_BIT_M_1 \& \sim bit_indexCMP7) \vee (\sim state[2] \& state[1] \& state[0] \& clk_countCMPCLKS_PER_BIT_M_1)$

Figura 5. Tabla de estados y entradas: UART_RX.

De un mismo modo, se establecieron los pertinentes diagrama de estados y tabla de estados y entradas para la máquina de estados finitos UART_Controller, como se observa en las figuras 6 y 7.

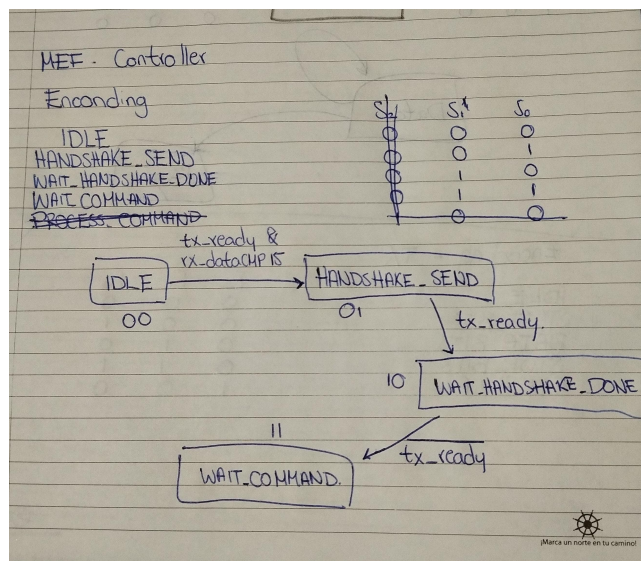


Figura 6. Diagrama de estados: UART_TX.

state[1]	state[0]	Inputs			next_state[1]	next_state[0]
		tx_ready	rx_ready	rx_dataCMP7		
0	0	1	1	1	0	1
0	0	0	X	X	0	0
0	0	X	0	X	0	0
0	0	X	X	0	0	0
0	1	1	X	X	1	0
0	1	0	X	X	0	1
1	0	0	X	X	1	1
1	0	1	X	X	1	0
1	1	X	X	X	1	1

$$\text{state_next}[1] = (\sim\text{state}[1] \& \text{state}[0] \& \text{tx_ready}) \vee (\text{state}[1] \& \sim\text{state}[0] \& \sim\text{tx_ready}) \vee (\text{state}[1] \& \sim\text{state}[0] \& \text{tx_ready}) \vee (\text{state}[1] \& \text{state}[0])$$

$$\text{state_next}[0] = (\sim\text{state}[1] \& \sim\text{state}[0] \& \text{tx_ready} \& \text{rx_ready} \& \text{rx_dataCMP7}) \vee (\sim\text{state}[1] \& \text{state}[0] \& \sim\text{tx_ready}) \vee (\text{state}[1] \& \sim\text{state}[0] \& \sim\text{tx_ready}) \vee (\text{state}[1] \& \text{state}[0])$$

Figura 7. Tabla de estados y entradas: UART_RX.

II-B. Decodificador para el visualizador de siete segmentos

Para la implementación del decodificador del siete segmentos que representa el resultado calculado por la ALU, se inició a partir de una tabla de verdad como la que se muestra en la figura, la cual representa diferentes números en su equivalente hexadecimal separado en siete salidas que representan los segmentos de la pantalla a utilizar.

	I ₁	I ₂	I ₃	I ₄	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	0	1	1	0	0	1	1	1	1
2	0	0	1	0	0	0	1	0	0	1	0
3	0	0	1	1	0	0	0	0	1	1	0
4	0	1	0	0	1	0	0	1	1	0	0
5	0	1	0	1	0	1	0	0	1	0	0
6	0	1	1	0	0	1	0	0	0	0	0
7	0	1	1	1	0	0	0	1	1	1	1
8	1	0	0	0	0	0	0	0	0	0	0
9	1	0	0	1	0	0	0	0	1	0	0
10	1	0	1	0	0	0	0	1	0	0	0
11	1	0	1	1	1	1	0	0	0	0	0
12	1	1	0	0	0	1	1	0	0	0	1
13	1	1	0	1	1	0	0	0	0	1	0
14	1	1	1	0	0	1	1	0	0	0	0
15	1	1	1	1	0	1	1	1	0	0	0

Figura 8. Tabla de verdad para el decodificador de siete segmentos.

Utilizando la metodología de los mapas de Karnaugh fue posible reducir la tabla de verdad anterior a sus siete salidas independientes y funcionales únicamente a partir del uso de compuertas lógicas, como se observa en la figura 9.

II-C. Fundamento y lógica del diseño PWM

Investigación 19/05/2025

A partir de la investigación realizada, se estableció que un generador PWM entrega una señal digital con frecuencia constante y ciclo de trabajo variable. Esta señal activa y desactiva el motor en intervalos definidos, permitiendo ajustar su velocidad. Para este proyecto, se requiere implementar 16 niveles de velocidad, lo que implica dividir el período completo en 16 subperíodos o ciclos de reloj.


```

○ ○ ○

segments[6] = (value[1] & value[0] & value[3] & ~value[2]) |
               (~value[1] & value[0] & value[3] & value[2]) |
               (~value[1] & value[0] & ~value[3] & ~value[2]) |
               (~value[1] & ~value[0] & ~value[3] & value[2])

segments[5] = (value[1] & ~value[0] & value[2]) |
               (value[1] & value[0] & value[3]) |
               (~value[0] & value[3] & value[2]) |
               (~value[1] & value[0] & ~value[3] & value[2])

segments[4] = (value[1] & value[3] & value[2]) |
               (~value[0] & value[3] & value[2]) |
               (value[1] & ~value[0] & ~value[3] & ~value[2])

segments[3] = (value[1] & value[0] & value[2]) |
               (value[1] & ~value[0] & value[3] & ~value[2]) |
               (~value[1] & value[0] & ~value[3] & ~value[2]) |
               (~value[1] & ~value[0] & ~value[3] & value[2])

segments[2] = (value[0] & ~value[3]) |
               (~value[1] & value[0] & ~value[2]) |
               (~value[1] & ~value[3] & value[2])

segments[1] = (value[1] & ~value[3] & ~value[2]) |
               (value[1] & value[0] & ~value[3]) |
               (value[0] & ~value[3] & ~value[2]) |
               (~value[1] & value[0] & value[3] & value[2])

segments[0] = (~value[1] & ~value[3] & ~value[2]) |
               (value[1] & value[0] & ~value[3] & value[2]) |
               (~value[1] & ~value[0] & value[3] & value[2])

```

Figura 9. Siete salidas del decodificador calculada mediante mapas de Karnaugh.

Desarrollo del código 21/05/2025 - 25/05/2025

Para dar solución al pwm se plantea el uso de un contador de 4 bits que recorra valores de 0 a 15. En cada ciclo, este contador se compara contra un valor llamado duty, el cual representa la velocidad deseada. Mientras el valor del contador sea menor al duty, la salida estará en alto, en caso contrario, en bajo. La señal pwmOut se define a partir de la resta entre counter y duty, usando un restador estructural. Si el resultado de la resta genera un bit de borrow, significa que counter > duty y por tanto la salida es 1.

Ecuación booleana de salida:

$$pwm_out = borrow = (counter < duty)$$

Componentes estructurales Utilizados:

- **Contador de 4 bits:** Implementado con un sumador estructural (adder_4bit) y flip-flops.
- **Comparador estructural:** Se implementa usando un restador de 4 bits (subtractor_4bit) y su bit de *borrow*.
- **Multiplexores estructurales:** Se usan para el reinicio del contador (rst) y para definir el valor efectivo del duty.

III. CÓDIGO FUENTE

https://github.com/CAMANEM/O_Soto_compu_archi_found_1SG1_2025

REFERENCIAS

- [1] David Money Harris and Sarah L. Harris. *Digital Design and Computer Architecture*. Morgan Kaufmann, 2nd edition, 2012.
- [2] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 5th edition, 2013.
- [3] Neil H. E. Weste and David Harris. *CMOS VLSI Design: A Circuits and Systems Perspective*. Pearson, 4th edition, 2010.